

পথচারীর রাস্তা পারাপার পূর্বাভাস – আমাদের গবেষণার পূর্ণ ব্যাখ্যা

শুরু থেকে শেষ পর্যন্ত, সহজ বাংলায়। প্রতিটি সংখ্যা repository-র ফলাফল ফাইল থেকে যাচাই করা।

Repository: [pedestrian-intention-temporal-validity](#) · Dataset: PIE, JAAD, IDD-PeD · সব যাচাই docs/bangla_walkthrough.html তৈরির সময় করা

এই নথিটা কী জন্য

এটা আমাদের পুরো কাজের গল্প – সুপারভাইজারকে ব্যাখ্যা করার জন্য এবং paper লেখার আগে নিজে পুরোপুরি বোঝার জন্য। কোনো দাবি অনুমান থেকে লেখা হয়নি; প্রতিটি সংখ্যার উৎস §১২ যাচাই লগ-এ দেওয়া আছে। সেখানে ৪টা গরমিলও সংভাবে লেখা আছে, যেগুলো যাচাই করতে গিয়ে ধরা পড়েছে।

সূচিপত্র

- | | |
|--|-------------------------------------|
| ১. আমরা কী নিয়ে কাজ করছিলাম | ৭. সমাধান ২ – Phase-Matched Control |
| ২. সমস্যা ১ – Temporal Contamination | ৮. ফলাফল কী বদলে গেল |
| ৩. সমাধান ১ – Event-Anchored Protocol | ৯. অন্য dataset-এ যাচাই |
| ৪. পরিষ্কার করার পর যা দেখলাম | ১০. Paper-এর গল্প |
| ৫. সমস্যা ২ – Class-Dependent Sampling Phase | ১১. সীমাবদ্ধতা – সং তালিকা |
| ৬. AUC = 1.0000 আমরা কীভাবে মাপলাম | ১২. যাচাই লগ ও গরমিল |

১. আমরা কী নিয়ে কাজ করছিলাম

আমরা **PIE dataset** নিয়ে কাজ করছিলাম – গাড়ির সামনে ক্যামেরা লাগানো, রাস্তায় পথচারীদের ভিডিও। লক্ষ্য: **একজন পথচারী রাস্তা পার হবে কি না, ঘটনার আগেই বলে দেওয়া।**

Model যা দেখে

- ১৬ ফ্রেমের একটা ছোট window (প্রায় আধা সেকেন্ড)
- প্রতি ফ্রেমে ৫টা সংখ্যা – পথচারীর bounding box [x1, y1, x2, y2] আর গাড়ির গতি vehicle_speed
- কাঁচা pixel স্থানাঙ্ক (1920×1080), image size দিয়ে ভাগ করা নয়

Data ভাগ (কখনো এলোমেলো ভাগ নয়)

Recording set অনুযায়ী স্থির ভাগ, যাতে একই পথচারী দুই ভাগে না পড়ে: **train = set01/02/04, val = set05/06, test = set03।**

Model ছিল একটা **BiLSTM**। ফলাফল আসছিল $AUC \approx 0.93$ – বেশ ভালো দেখাচ্ছিল। সমস্যা শুরু হলো যখন আমরা প্রশ্ন করলাম – আসলেই কি model ভবিষ্যদ্বাণী করছে, নাকি অন্য কিছু করছে?

২. সমস্যা ১ — Temporal Contamination (সময়গত দূষণ)

সমস্যাটা কী

যাচাই করে দেখলাম, যে ১৬ ফ্রেমের window দেখে model "ভবিষ্যদ্বাণী" করছে — সেই window-এর ভেতরেই পথচারী ইতিমধ্যে রাস্তা পার হওয়া শুরু করে ফেলেছে।

উপমা: পরীক্ষার আগে প্রশ্নপত্র ফাঁস হয়ে যাওয়া। ছাত্র ভালো নম্বর পাচ্ছে, কিন্তু সেটা তার জ্ঞানের প্রমাণ নয়। Model ভবিষ্যদ্বাণী করছে না — সে উত্তরটা দেখে ফেলেছে।

আমরা কতটুকু মেপেছি

Dataset	crossing window মোট	দূষিত	শতাংশ
PIE (পুরনো track-end anchor)	570	387	67.9%
JAAD (naive anchor)	460	428	93.0%
IDD-PeD (naive anchor)	—	—	81.3%

PIE-তে 369টা window (crossers-এর 64.7%) ছিল পুরোপুরি crossing-এর ভেতরে — মানে ঐগুলোতে কোনো ভবিষ্যদ্বাণীর প্রশ্নই ওঠে না, ওটা নিছক শনাক্তকরণ।

এটা হলো কেন

PIE-র annotation-এ দুইটা আলাদা জিনিস আছে:

- crossing — পথচারী এখন পার হচ্ছে কি না
- crossing_point — পার হওয়া শুরু হয়েছে কোন ফ্রেমে

প্রচলিত code শুধু crossing পড়ে, crossing_point কখনো পড়ে না। তাই window গুলো track-এর শেষ থেকে গোনো হয় — এবং crossing-এর উপর গিয়ে পড়ে।

গুরুত্বপূর্ণ: এটা কারো bug নয় — এটা পুরো field-এর একটা গঠনগত ভুল। এজন্যই এটা paper-এর যোগ্য। ধারণাটা PCPA (WACV 2021) আগেই নাম দিয়েছে; কিন্তু কেউ কখনো এর পরিমাণ মাপেনি — সেটাই আমাদের অবদান।

বাইরের সমর্থন

GTransPDM (IEEE SPL 2025), Fig. 4 — তারা TTE ব্যবধান কমাতে কমাতে দেখেছে: "TTE = 0-তে accuracy ও F1 পৌঁছায় 99.32% ও 98.73%।" মানে ব্যবধান শূন্য হলেই কাজটা প্রায় নিখুঁতভাবে সমাধানযোগ্য হয়ে যায়। ওই paper-এর এই তর্কে কোনো পক্ষ নেই — তবু তারা একই ঘটনা দেখাচ্ছে। আমাদের audit ব্যাখ্যা করে কেন।

৩. সমাধান ১ — Event-Anchored Protocol

আমরা window বানানোর নিয়ম বদলে দিলাম।

	নিয়ম
পুরানো	track-এর শেষ থেকে পিছিয়ে গিয়ে window নাও
নতুন	crossing_point-এ anchor করো, তারপর window বসাও তার ৩০-৬০ ফ্রেম আগে

এতে crossing window-এ ঢোকা গঠনগতভাবেই অসম্ভব হয়ে যায়।

আমরা দুইভাবে যাচাই করেছি

- ফলাফল পরিষ্কার কি না: নতুন dataset-এ 4,906 window-এর মধ্যে 0টা দূষিত — কোনো window-এ একটাও crossing ফ্রেম নেই।
- পুনরুৎপাদনযোগ্য কি না: কাঁচা PIE থেকে dataset-টা ভুবু (bit-exact) আবার তৈরি হয়।

পরিষ্কার dataset-এর গঠন

	window	পথচারী
মোট	4,906	1,267
Positive (পার হয়)	1,648	434
Negative (পার হয় না)	3,258	833

৪. পরিষ্কার করার পর যা দেখলাম

(ক) "সহজ shortcut" অনেকটা গায়েব হয়ে গেল

আগে model শুধু box-এর আকার দেখেই উত্তর দিতে পারত (কাছে এলে box বড় হয়)। নিচে rank-biserial মান — 0 মানে কোনো পার্থক্য নেই, 0.5 মানে খুব বড় পার্থক্য।

সংকেত	আগে (leaky)	পরে (clean)
box area	+0.654	+0.247
box height	+0.629	+0.211
box-এর নিচের কিনারা	+0.485	+0.090
box-এর x-কেন্দ্র	-0.186	-0.018 (তাৎপর্যহীন)

মানে: box geometry দিয়ে শ্রেণী আলাদা করার সহজ পথটা leakage-এরই একটা উপসর্গ ছিল।

(খ) চারটা model family — একই নিয়মে তুলনা

আমরা চারটা family তুলনা করলাম — **BiLSTM, Transformer, GRU, আর un-gated vanilla RNN** — সবাইকে একই engine, একই device (CPU), একই ৫টা seed, একই class weight (1.682), একই checkpoint নিয়ম দিয়ে। পরিসংখ্যান: পথচারী-ভিত্তিক bootstrap (B = 10,000) + Holm সংশোধন, ৩০টা তুলনা।

Model	Parameter	AUC	F1
Vanilla RNN	560,001	0.9481	0.8487
Transformer	794,241	0.9447	0.8250
GRU	1,678,209	0.9375	0.8419
BiLSTM-h128	594,561	0.9349	0.8256
BiLSTM	2,237,313	0.9242	0.8276

৩০টা তুলনার মধ্যে **৯টা** Holm সংশোধনের পরেও টিকে যায় (সংশোধন ছাড়া ১৮টা)। সবচেয়ে ছোট model — Vanilla RNN — সবার উপরে; সে Transformer-এর সাথে **সমান** ($\Delta\text{AUC} +0.0006$, $p = 0.83$)।

(গ) সবচেয়ে গুরুত্বপূর্ণ — একটা সাধারণ Logistic Regression চারটা neural model-এর সমান

আমরা একটা **খুব সাধারণ logistic regression** চালানো — কোনো neural network নয়, শুধু ৮০টা কাঁচা সংখ্যার (১৬ ফ্রেম $\times$ ৫ feature) উপর একটা সরলরৈখিক model।

Baseline	AUC	PR-AUC	F1
LR — bbox + গাড়ির গতি (৮০ feature)	0.9488	0.9121	0.8546
LR — শুধু গাড়ির গতি (১৬ feature)	0.9335	0.8538	0.8199
LR — শুধু একটা ফ্রেম (৫ feature)	0.9251	0.8757	0.7903
LR — শুধু bbox (৬৪ feature)	0.9129	0.8035	0.7812
সবাইকে "হ্যাঁ" বলে দেওয়া	0.500	0.3252	0.4908

সেই neural model-এর সাথে পার্থক্য পরিসংখ্যানগতভাবে শূন্য:

AUC $p = 0.270$ · PR-AUC $p = 0.773$ · F1 $p = 0.447$

তিনটাই পথচারী-ভিত্তিক bootstrap, B = 10,000

শুধু **গাড়ির গতি** দিয়ে (১৬টা সংখ্যা, সরলরৈখিক) — AUC 0.9335, যা 2.24M parameter-এর BiLSTM-এর (0.9242) চেয়েও বেশি। শুধু **একটা ফ্রেম** দিয়ে — 0.9251।

কেন এটা নতুন: এই field-এ কেউ কখনো এত সাধারণ baseline report করেনি। PCPA-র সবচেয়ে দুর্বল তুলনাও একটা গভীর CNN। majority-class, শুধু-গতি, বা কাঁচা window-এর উপর সরলরৈখিক model – কোনো paper-এ নেই।

কেন এটা গুরুত্বপূর্ণ: এটা এই literature-এর সব architecture-দাবির একটা সীমা টেনে দেয় – আমাদের নিজেদের দাবিসহ।

৫. সমস্যা ২ — Class-Dependent Sampling Phase

এখানেই গল্পটা সবচেয়ে আকর্ষণীয় হয়।

প্রথমে: Negative class কেন লাগে?

একটা স্বাভাবিক প্রশ্ন — "যারা পার হবে না তাদের নিয়ে ভাবছি কেন?"

ভাবুন — যদি dataset-এ শুধু crossing পথচারী থাকত, model কী শিখত? সে শিখত "সবাইকে হ্যাঁ বলে দাও" — ১০০% accuracy, কিন্তু সম্পূর্ণ অকেজো।

মূল কথা: Model "crossing behaviour" শেখে না। Model শেখে — "যে দুইটা দল আমাকে দেখানো হয়েছে, তাদের মধ্যে পার্থক্যটা কী?"

এবং সে সবচেয়ে সহজ পার্থক্যটা ধরে। পার্থক্যটা অর্থপূর্ণ কি না, তার কোনো মাথাব্যথা নেই।

আমাদের 4,906 window-এর মধ্যে **3,258টাই negative** — অর্ধেকের বেশি। এরাই model-কে বলে দেয় "না" জিনিসটা দেখতে কেমন। **ওরা ভুল হলে model-ও ভুল।**

Negative-দের window আসলে কোথা থেকে আসে?

Positive-দের জন্য সহজ — crossing একটা আসল ঘটনা, ওটার ৩০-৬০ ফ্রেম আগে window বসাই।

কিন্তু negative-দের? তাদের তো কোনো ঘটনাই নেই! তাহলে window কোথায় বসাব?

প্রচলিত code (PIE-র নিজের extract_tracks_tte) যা করে: **track-এর শেষ ফ্রেম - ২**। আমরা মেপে দেখেছি — যে ৮৩৩ জন non-crossing পথচারী পরিষ্কার dataset-এ আছে, তাদের **৮২৪ জনের (৯৮.৯%)** ক্ষেত্রে এই দূরত্ব ঠিক ২ ফ্রেম।

এখন প্রশ্ন করুন — **একজন পথচারীর annotation track শেষ হয় কখন?** যখন সে ক্যামেরা থেকে হারিয়ে যায়। আর সে হারিয়ে যায় কখন? **যখন গাড়ি তাকে পাশ কাটিয়ে চলে যায়।**

ফলে একটা দুর্ঘটনাজনিত পার্থক্য তৈরি হয়ে গেল

শ্রেণী	window কখন নেওয়া হয়েছে	track শেষ হতে বাকি (ফ্রেম)
Negative	গাড়ি পাশ কাটানোর ঠিক ১-২ সেকেন্ড আগে	৩২ — ৬২ (মধ্যক ৪৬)
Positive	crossing-এর আগে, গাড়ি পৌঁছানোর অনেক আগে	৮৮ — ৬,৬০৬ (মধ্যক ২৮৭)

দুইটার মধ্যে একটাও overlap নেই। শূন্য।

সবচেয়ে বড় negative = ৬২। সবচেয়ে ছোট positive = ৮৮। **৬২ < ৮৮।**

আমরা যাচাই করেছি: 3,258টা negative-এর মধ্যে **০টা** positive-দের সর্বনিম্নের উপরে।

এটা leakage থেকে আলাদা কেন — এই পার্থক্যটা মনে রাখতে হবে

	Temporal Leakage	Class-Dependent Sampling Phase
সমস্যা	window-এ উত্তরটা দেখা যায়	window-এ উত্তর নেই, কিন্তু কখন তোলা হয়েছে সেটাই উত্তর ফাঁস করে
কাকে প্রভাবিত করে	positive class	negative class
আমাদের clean protocol ধরে?	হ্যাঁ — 0/4906	না — একটুও না

এটাই আমাদের paper-এর সবচেয়ে বড় কথা।

আমরা leakage সত্যিই ঠিক করেছি — 4,906টার একটাতেও crossing নেই, যাচাই করা। কিন্তু তারপরেও data-তে AUC 1.0000-এর একটা লুকানো সংকেত রয়ে গেছে।

মানে: যে সমাধানটা সবাই যথেষ্ট মনে করবে, সেটা যথেষ্ট নয়।

Model এটা কীভাবে ধরে ফেলে?

Model তো "কত ফ্রেম বাকি" দেখে না — সে শুধু ৫টা সংখ্যা দেখে। কিন্তু গাড়ি যখন কারো পাশ কাটানোর ১-২ সেকেন্ড আগে থাকে, তখন:

- তার bounding box খুব দ্রুত বড় হচ্ছে (কাছে চলে এসেছে)
- box ফ্রেমের কিনারার দিকে সরছে
- গাড়ির গতির একটা নির্দিষ্ট ধরন থাকে

Model এগুলো দেখে পরোক্ষভাবে ওই লুকানো সংকেতটাই শিখে ফেলে। সে ভাবছে না "এই লোক পার হবে না" — সে ভাবছে "এই recording এক্ষুনি শেষ হয়ে যাবে"।

উপমা: আপনি ছবি দেখে বলতে চান কোন ছাত্র পাশ করবে।

পাশ করা ছাত্রদের ছবি তুললেন পরীক্ষার ৬ মাস আগে, ক্লাসরুমে।

ফেল করা ছাত্রদের ছবি তুললেন তাদের কলেজ ছাড়ার শেষ দিনে, গেটের সামনে ব্যাগ হাতে।

Model ৯৫% accuracy পাবে। কিন্তু সে শিখেছে "গেটের সামনে ব্যাগ হাতে = ফেল" — পড়াশোনা নয়। বাস্তবে চালালে সম্পূর্ণ ব্যর্থ হবে।

৬. AUC = 1.0000 আমরা কীভাবে মাপলাম

প্রথমে একটা ভুল ধারণা সরাতে হবে

অনেকে ভাবেন AUC মানেই model-এর স্কোর। আসলে তা নয়।

AUC-র জন্য কোনো model লাগে না। AUC শুধু মাপে – "একটামাত্র সংখ্যা দিয়ে দুইটা দলকে কতটা আলাদা করা যায়?"

তাই আমরা **কোনো model train করিনি**। শুধু একটা সংখ্যা নিয়ে প্রশ্ন করেছি: "এই একটা সংখ্যাই কি উত্তর বলে দিতে পারে?"

যে সংখ্যাটা নিয়েছি

```
to_end = (track-এর শেষ ফ্রেম) - (window-এর anchor ফ্রেম)
```

মানে: "এই window-টার পর ওই পথচারী আর কত ফ্রেম ক্যামেরায় ছিল?"

AUC আসলে যা মাপে

সংজ্ঞাটা খুব সহজ: **এলোমেলোভাবে একটা positive আর একটা negative তুললে – positive-এর সংখ্যাটা কত শতাংশ ক্ষেত্রে বড় হয়?**

ছোট উদাহরণ

ধরুন মাত্র ৩ জন positive (to_end = 100, 250, 400) আর ৩ জন negative (40, 50, 60)। সব জোড়া মিলিয়ে দেখুন – মোট $3 \times 3 = 9$ টি জোড়া:

100>40 ✓	100>50 ✓	100>60 ✓
250>40 ✓	250>50 ✓	250>60 ✓
400>40 ✓	400>50 ✓	400>60 ✓

→ AUC = 9/9 = 1.0000

আমাদের আসল data-তে ঠিক এটাই

Negative সবাই ৩২-৬২, positive সবাই ৮৮-৬,৬০৬। তাই **প্রত্যেকটা positive প্রত্যেকটা negative-এর চেয়ে বড়**। জোড়ার সংখ্যা $1,648 \times 3,258 = 53$ লাখেরও বেশি – এবং সবগুলোই সঠিক।

স্বাধীনভাবে পুনঃযাচাই করা: sklearn.metrics.roc_auc_score দিয়ে সরাসরি হিসাব করে পেয়েছি – মূল পরিস্কার data-তে AUC(to_end) = 1.000000, control-এর পর 0.777941।

এটাই diagnostic। আমরা প্রমাণ করলাম – model ছাড়াই, শুধু "ক্যামেরা আর কতক্ষণ চলবে" জানলেই ১০০% নিখুঁত উত্তর দেওয়া যায়।

৭. সমাধান ২ — Phase-Matched Control

মূল ধারণা এক লাইনে: একই পথচারী, একই ভিডিও, একই আচরণ — শুধু তাকে আমরা আগে গিয়ে দেখব।

আমরা কোনো নকল data বানাইনি, negative class বাদ দিইনি। শুধু **কখন তাকাচ্ছি** সেটা বদলেছি।

ধাপে ধাপে

ধাপ ১ — Positive-দের to_end জমা করলাম

1,648টা positive window-এর মান একটা তালিকায় রাখলাম। এটাই আমাদের **লক্ষ্য-বস্টন**।

ধাপ ২ — Positive-দের কিছু করলাম না

তাদের সময় crossing ঘটনা দিয়ে বাঁধা। **হুবহু আগের window রেখে দিলাম**।

ধাপ ৩ — প্রত্যেক negative পথচারীর জন্য নতুন সময় বাছলাম

```
f0, fN = track-এর প্রথম ও শেষ ফ্রেম
hi      = fN - f0 - 15                                # সর্বোচ্চ কত পিছনে যাওয়া সম্ভব

if hi < 88:                                            # track খুব ছোট
    বাদ দাও

target = positive তালিকা থেকে এলোমেলোভাবে একটা মান
te      = clip(target, 88, hi)                        # যতটুকু সম্ভব
anchor = fN - te                                       # শেষ থেকে te ফ্রেম পিছনে
window = ওখানকার ১৬টা পরপর ফ্রেম
```

৮৮ সংখ্যাটা এলোমেলো নয় — **এটা positive-দের সর্বনিম্ন to_end**, অর্থাৎ যেখান থেকে দুই শ্রেণীর একসাথে থাকা সম্ভব তার সীমানা।

একটা বাস্তব উদাহরণ

ধরুন **রহিম** একজন negative পথচারী, তার track ফ্রেম 1000 থেকে 1500।

পুরনো: 1000 ————— 1500
anchor 1454 (শেষ থেকে ৪৬ ফ্রেম)

positive তালিকা থেকে এলোমেলোভাবে উঠল 250 → নতুন anchor = 1500 - 250 = 1250

নতুন: 1000 ————— 1500
anchor 1250 (শেষ থেকে ২৫০ ফ্রেম)

রহিম একই মানুষ। একই ভিডিও। সে এখনো রাস্তা পার হয় না। শুধু আমরা তাকে ২০০ ফ্রেম আগে দেখছি — ঠিক যতটা আগে crossing-দের দেখি।

ফলাফল

	Negative to_end	Positive to_end
আগে	৩২ – ৬২ (মধ্যক ৪৬)	৮৮ – ৬,৬০৬ (মধ্যক ২৮৭)
পরে	৮৮ – ১,৩২১ (মধ্যক ১৮৮)	৮৮ – ৬,৬০৬ (অপরিবর্তিত)

- **2,872 / 2,872** negative এখন positive-দের সর্বনিম্ন (৮৮)-এর উপরে
- **1,619 / 1,648** positive এখন negative-দের পরিসরের ভেতরে

আগে overlap ছিল **শূন্য**। এখন **প্রায় সম্পূর্ণ**। এজন্যই লুকানো সংকেত **1.0000 → 0.7779**।

দাম কী দিলাম

	পরিষ্কার data	Phase-matched
মোট window	4,906	4,520
Positive	1,648	1,648 (অপরিবর্তিত)
Negative	3,258	2,872
Negative পথচারী	833	718 (১১৫ বাদ)
pos_weight	1.682	1.5665

0.5000 না হয়ে 0.7779 কেন?

সং উত্তর: **আমরা সমস্যাটা কমিয়েছি, পুরোপুরি মুছিনি।**

কারণ clip। **Negative-দের track এমনতেই ছোট।** ধরুন positive তালিকা থেকে উঠল ৮০০, কিন্তু রহিমের পুরো track-ই মাত্র ৪০০ ফ্রেম। তখন বাধ্য হয়ে ৪০০-তে কেটে দিতে হয়।

তাই negative-রা positive-দের **লম্বা লেজটা ছুঁতে পারে না** – negative সর্বোচ্চ ১,৩২১, positive সর্বোচ্চ ৬,৬০৬। মধ্যকেও পার্থক্য থাকে: ১৮৮ বনাম ২৮৭। **এই অবশিষ্ট পার্থক্যটাই 0.7779।**

আরও কমানো যেত? হ্যাঁ, কিন্তু দাম দিয়ে – MIN_TO_END ৮৮ থেকে ১৫৮ করলে আরও প্রায় ৪০% negative বাদ পড়ত; positive-দের track কেটে ছোট করলে তাদের সময়ও কৃত্রিম হয়ে যেত।

৮. ফলাফল কী বদলে গেল

এইটাই সবচেয়ে গুরুত্বপূর্ণ — এবং সবচেয়ে সং — অংশ।

(ক) সব model-এর স্কোর পড়ে গেল

Model	AUC আগে	AUC control-এ	Δ	F1 আগে	F1 control-এ	Δ
BiLSTM	0.9242	0.8842	-0.040	0.8276	0.7644	-0.063
Transformer	0.9447	0.8928	-0.052	0.8250	0.7609	-0.064
GRU	0.9375	0.8828	-0.055	0.8419	0.7642	-0.078
Vanilla RNN	0.9481	0.8852	-0.063	0.8487	0.7531	-0.096
Logistic Regression (৮০)	0.9488	0.9031	-0.046	0.8546	0.7561	-0.099

মানে আগের 0.93-0.95-এর একটা বড় অংশ ছিল model-এর দক্ষতা নয় — dataset-এর দুর্ঘটনা।

(খ) আমাদের নিজেদের "সেরা model" আর সেরা রইল না

আগে **Vanilla RNN** সবার সেরা ছিল। Control-এর পর সে AUC-তে **দ্বিতীয়**, আর F1-এ **সবার শেষে** (0.7531)। এবং সে-ই সবচেয়ে বেশি নেমেছে (-0.063 AUC, -0.096 F1)।

এর মানে — RNN-এর জেতাটা মূলত ওই লুকানো সংকেত ব্যবহার করেই হচ্ছিল।

Control-এ চারটা family-র AUC ছড়ানো মাত্র **0.0100** (0.8828 - 0.8928), অথচ প্রতিটি family-র seed-ভিত্তিক SD **0.0113 - 0.0125**। মানে পার্থক্যটা এক SD-রও কম — ranking কার্যত মিলিয়ে যায়।

সতর্কতা: control arm-এ এখনো Holm/bootstrap চালানো হয়নি। তাই উপরের কথাটা **point estimate**-এর পর্যবেক্ষণ, **পরীক্ষিত** দাবি নয়। এটা চালানো এখনো বাকি — একটা ছোট কাজ, checkpoint গুলো ইতিমধ্যেই সংরক্ষিত।

(গ) গাড়ির গতির গল্প উল্টে গেল

	আগে	control-এ	Δ
LR — শুধু গাড়ির গতি	0.9335	0.8257	-0.108
LR — শুধু bounding box	0.9129	0.8987	-0.014

কারণটা যুক্তিসঙ্গত: গাড়ির গতি হলো "গাড়ি এই লোককে কতটা কাছ থেকে পাশ কাটাতে যাচ্ছে" — এর সবচেয়ে সরাসরি পরিমাপ। তাই timing bias সরানোর পর সে তার শক্তির বড় অংশ হারায়। Bounding box পথচারীর **আসল** কিছু তথ্য বহন করে — সে কোথায় দাঁড়িয়ে, ফুটপাথের কিনারায় কি না, কোন দিকে সরছে। তাই সে প্রায় অটুট থাকে।

আমরা এতদিন ভাবছিলাম "গাড়ির গতিই সবচেয়ে শক্তিশালী সংকেত"। **Control দেখাল — ওটার বড় অংশ ছিল sampling-এর কৃত্রিম ফল, পথচারীর আচরণ নয়।**

(ঘ) Logistic Regression এখন আর সমান নয় — সে জিতে যায়

Control-এ LR (AUC 0.9031) চারটা neural model-কেই হারায় (সেরা neural 0.8928)। PR-AUC-তে ব্যবধান আরও বড়: 0.8184 বনাম 0.7821।

মানে আমাদের §৪(গ)-এর দাবিটা control-এ দুর্বল হয় না, বরং আরও শক্তিশালী হয়।

(ঙ) ঘটনাক্রমে: 5-D বনাম 4-D ablation

গাড়ির গতি বাদ দিলে কী হয় — family অনুযায়ী খুব আলাদা:

Family	5-D AUC	4-D AUC	পার্থক্য	Holm-এ টেকে?
BiLSTM	0.9242	0.7765	+0.1477	হ্যাঁ
Vanilla RNN	0.9481	0.8766	+0.0715	হ্যাঁ
GRU	0.9375	0.8836	+0.0538	হ্যাঁ
Transformer	0.9447	0.9291	+0.0156	না

এখানে একটা ফাঁদ আছে — paper-এ সাবধান। BiLSTM-এর 4-D স্কোর (0.7765) দেখে মনে হয় "গাড়ির গতি ছাড়া BiLSTM পারে না"। কিন্তু **একই bbox-only input-এ logistic regression পায় 0.9129** — চারটার মধ্যে তিনটার উপরে। তার মানে তথ্যটা data-তে আছে, **BiLSTM সেটা বের করতে ব্যর্থ হচ্ছে**। এটা architecture-এর ক্ষমতার ক্রম নয়, একটা **fitting failure**।

৯. অন্য dataset-এ যাচাই

JAAD

একই audit চালিয়ে: crossers-এর **93.0%** (428/460) দূষিত, সব sequence মিলিয়ে 81.3%। সমস্যাটা PIE-র একার নয়। Box geometry shortcut-ও একইভাবে ভেঙে পড়ে (area +0.478 → সব $|rb| < 0.17$)।

IDD-PeD (ভারতের অগোছালো ট্রাফিক)

Anchor	window	দূষণ
naive (track-end)	8,651	81.3% of crossers
crossing_point (PIE-র নিয়ম ছবছ)	7,919	29.6% of crossers
strict — min(cp, onset)	7,318	0.0%

এটা একটা স্বতন্ত্র অবদান: PIE-তে যে নিয়ম 0%-এ নামায়, IDD-PeD-তে সেটা **29.6% দূষণ রেখে দেয়** — কারণ ওদের crossing_point নির্ভরযোগ্য onset চিহ্ন নয়। মানে সমাধান দেশান্তরে ছবছ বসে না।

এটা IDD-PeD-র নিজের paper-এর একটা স্পষ্ট দাবির **বিপরীত** (তারা লিখেছে সব crossing case-এ observation ও TTE দুটোই crossing-এর আগে)। Paper-এ এটা অভিযোগ হিসেবে নয়, তাদের প্রকাশিত ফাইল থেকে পুনরুৎপাদনযোগ্য পর্যবেক্ষণ হিসেবে লিখতে হবে। এটাই সবচেয়ে বিতর্কিত দাবি হবে বলে ধরে নেওয়া উচিত।

Zero-shot transfer — PIE-তে শেখা model IDD-PeD-তে

Test prevalence 7.13%, তাই "সবাইকে হ্যাঁ" বললে F1 = **0.1331**।

Model	IDD AUC	IDD F1	PIE AUC
BiLSTM-F1	0.675	0.1311	0.940
Transformer-F1	0.698	0.1296	0.947
GRU-F1	0.713	0.1291	0.941
Vanilla RNN-F1	0.720	0.1296	0.948

চারটাই trivial baseline-এর নিচে। শুধু ranking ক্ষমতা (AUC) কিছুটা টিকে থাকে।

সততার শর্ত: উপরের সংখ্যা pre-registered rescale স্থানান্তর-রূপান্তরের। প্রকাশিত sensitivity check raw-তে F1 0.244-0.268 — যা baseline-এর উপরে। raw-কে ইচ্ছাকৃতভাবে প্রধান ধরা হয়নি, কারণ IDD-PeD-র test AUC দেখে রূপান্তর বাছাই করা মানে test-set selection। **Paper-এ কোনটা quote করছেন তা স্পষ্ট বলতে হবে।**

মোতায়েন করা model বাস্তবে কী করে

Demo-র ভবিষ্যদ্বাণী ফাইল থেকে (19,799 রেকর্ড, 8টা ভিডিও অংশ) নিজে হিসাব করে পেয়েছি:

- গাড়ির গতি ও ভবিষ্যদ্বাণীর সম্পর্ক **Pearson r = -0.892**
- গাড়ি থেমে থাকলে (0 km/h) **96.2%** পথচারীকে "পার হবে" বলে
- ২০ km/h-এর উপরে মাত্র **1.00%**

মানে মোতায়েন করা system পথচারীদের মধ্যে আলাদা করে বিচার প্রায় করছেই না — **সে কার্যত একটা গতির থ্রেশহোল্ড**। এটা "input signal-ই প্রধান" কথাটাকে সমর্থন করে, এবং Diving Deeper (IV 2024) Table IV-এর সাথেও মেলে।

১০. Paper-এর গল্প

এই field-এর benchmark-এ একটা নয়, দুইটা মাপজোখের সমস্যা আছে।

প্রথম — Temporal contamination: যে ঘটনা predict করতে বলা হচ্ছে, সেটা crossing window-এর **67.9%** ক্ষেত্রে input-এই দেখা যাচ্ছে।

দ্বিতীয় — Class-dependent sampling phase: negative-দের সবসময় track শেষ হওয়ার ঠিক আগে দেখা হয়, যা একাই **AUC 1.0000** দিয়ে শ্রেণী আলাদা করে।

আসল কথা: প্রথমটার সমাধান দ্বিতীয়টাকে ছোঁয়ও না। দুইটাই নিয়ন্ত্রণ করলে সব model প্রায় **0.05 AUC** হারায়, গাড়ির গতির প্রাধান্য **উল্টে যায়**, চারটা architecture-এর পার্থক্য **এক seed SD-র ভেতরে** মিলিয়ে যায়, আর একটা সাধারণ **logistic regression** চারটা **neural network-কেই হারিয়ে দেয়**।

এটা "আমরা ভালো model বানিয়েছি" ধরনের paper নয়। এটা "এই benchmark যা মাপে বলে দাবি করে, তা মাপে না" ধরনের paper।

Paper-এ contribution গুলো কীভাবে সাজাবেন

অবদান	অবস্থা
দুষ্প্রের পরিমাণ মাপা (৩টা dataset)	মেরুদণ্ড control-এ অক্ষত
Event-anchored protocol (0/4906, bit-exact)	মেরুদণ্ড অক্ষত
Geometry shortcut = leakage-এর উপসর্গ	মেরুদণ্ড অক্ষত
Linear baseline সব architecture-দাবির সীমা	শক্তিশালী হয় control-এ LR জিতে যায়
Sampling-phase আবিষ্কার + control	নতুন মূল অবদান
গাড়ির গতির প্রাধান্য	পুনর্লিখন লাগবে ক্রম উল্টে যায়
"Vanilla RNN সেরা"	নামিয়ে আনতে হবে control-এ টেকে না
Cross-dataset (JAAD, IDD-PeD)	অক্ষত

"Vanilla RNN সেরা" দাবির বদলে যেটা লিখবেন, সেটা আসলে বেশি শক্তিশালী:

"এই benchmark-এ architecture-এর ক্রম স্থিতিশীল নয় – negative sampling-এ এমন একটা পরিবর্তন, যা কাজের সংজ্ঞা বদলায় না, তাতেই প্রথম স্থানের family দ্বিতীয়/শেষ হয়ে যায়।"

এটা একটা পদ্ধতিগত ফলাফল, নিছক একটা leaderboard নয়। আর Chung (2014)-এর বিরুদ্ধে "RNN জিতেছে" দাবিটা এমনভাবেও টিকত না।

১১. সীমাবদ্ধতা – সং তালিকা

এগুলো paper-এর Limitations-এ যাবে। কিন্তু §৮-এর সংখ্যাগুলো Results-এ যাবে, Limitations-এ নয়।

Control-এর নিজের সীমা

1. একটাই seed (SEED = 42) দিয়ে negative আবার তোলা হয়েছে – পুনঃনমুনায়নের ভিন্নতা মাপা হয়নি।
2. অবশিষ্ট separability 0.7779, 0.5 নয় – কমেছে, শেষ হয়নি।
3. ১১৫ জন negative পথচারী বাদ পড়েছে, তাই population একটু বদলেছে (test set 541 → 476 পথচারী)।
4. Control arm-এ significance test হয়নি – এগুলো এখনো শুধু point estimate।
5. একটা বিকল্প ব্যাখ্যা বাদ দেওয়া যায়নি: হয়তো নতুন negative গুলো এমনিতেই কঠিন (দূরে, ছোট box) – timing নয়, দূরত্বই কারণ। এটা আলাদা distance-matched design ছাড়া আলাদা করা যায় না।

মূল কাজের সীমা

6. কোনো equivalence margin আগে থেকে ঠিক করা হয়নি। "CI-তে শূন্য আছে" মানে "সমান" নয় – সেজন্য আগে থেকে ঠিক করে TOST লাগে (Bouthillier, MLSys 2021)।
7. Power রিপোর্ট করা হয়নি। এই নমুনা আকারে "তাৎপর্যপূর্ণ নয়" ফলাফল স্বাভাবিক – প্রতিটা null পড়তে হবে "আমরা পার্থক্য ধরতে পারিনি" হিসেবে, "পার্থক্য নেই" হিসেবে নয়।
8. Transformer-এর AUC সুবিধা capacity-matched নয় – 794,241 বনাম BiLSTM-এর 594,561 (1.34×), আর search budget 2.2×।
9. Validation ছোট এবং বহুবার ব্যবহৃত – 634 window / 164 পথচারী, চারটা পরপর নির্বাচনে + threshold ফিটিংয়ে একই set। তাই সব validation সংখ্যা উপরের সীমা।
10. Label track-স্তরের – একজন পথচারীর পুরো track-এর একটাই label, প্রতিটা window-এ ছড়ানো।
11. একটাই test split (set03), এবং দিনের আলো, শুকনো আবহাওয়া, একটাই শহর।
12. Cell-type সমতা মূলত Chung (2014)-এর পুনরাবৃত্তি – তিনি আগেই দেখিয়েছেন ছোট sequence-এ un-gated RNN gated cell-এর সমান। ১৬ ফ্রেম ঠিক সেই ছোট-দিগন্ত অঞ্চল।

শিরোনাম নিয়ে দুটো পরামর্শ

- "intention" শব্দটার বদলে **crossing action / outcome** লেখা বেশি সঠিক – কারণ label আসলে PIE-র crossing attribute, উদ্দেশ্য নয়।
- শিরোনামে "statistically rigorous" দাবি না করাই ভালো – Gesnouin (IV 2022) ইতিমধ্যেই Friedman + Wilcoxon-Holm ব্যবহার করেছে, BiPed ২০ initialisation-এর SD দেয়, MFT সব metric-এ ± দেয়। **যা সত্যিই খালি আছে তা সংকীর্ণ:** পথচারী-ভিত্তিক clustered resampling, equivalence margin, এবং তুলনীয় family-দের মধ্যে search budget সমান রাখা।

১২. যাচাই লগ ও গরমিল

এই নথির প্রতিটা সংখ্যা repository-র ফলাফল ফাইল থেকে মিলিয়ে দেখা হয়েছে। নিচে কোনটা কোথা থেকে, এবং **যে ৪টা গরমিল ধরা পড়েছে**।

স্বাধীনভাবে পুনরায় হিসাব করা (নতুন করে চালানো)

যা যাচাই করেছি	ফল
AUC(to_end) মূল পরিষ্কার data-তে sklearn	1.000000
AUC(to_end) phase-matched data-তে sklearn	0.777941
মূল to_end পরিসর – negative / positive	32–62 / 88–6606
Overlap (negative $\geq$ positive-এর সর্বনিম্ন)	0 / 3,258
Phase-matched to_end – negative	88–1,321 (মধ্যক 188)
Negative পথচারী: পরিষ্কার $\rightarrow$ matched	833 $\rightarrow$ 718
PIE-র সব পথচারী (annotation-এ)	1,374 = 855 না-পার + 519 পার
last_frame – crossing_point non-crossers-এ	824/833 ঠিক ২
Holm-এ টেকা তুলনা	9 / 30
Demo: Pearson r, 0 km/h flag হার	-0.892, 96.2%

ফলাফল ফাইল থেকে মিলিয়ে নেওয়া

results/leakage/pie_legacy_leakage_report.md — 387/570, 369 (64.7%), rank-biserial ·
results/leakage/pie_clean_leakage_report.md — 0/4906, 1648/3258 ·
results/cross_dataset/jaad_naive_leakage_report.md — 428/460 ·
results/cross_dataset/idd_results/table2_temporal_audit.csv — 81.3/29.6/0.0 · table4_zero_shot.csv — AUC ও F1
· experiments/02_model_comparison/trivial_baselines_results.json — সব LR সংখ্যা ও p-মান ·
matched_comparison_results.json — চর family, parameter, Holm · phase_matched_results.json — control-এর সব
ফলাফল · ego_speed_ablation_results.json — 5-D vs 4-D

যে ৪টা গরমিল ধরা পড়েছে

১. বাদ পড়া পথচারী: ১১৪ নাকি ১১৫?

Repository-র নথি (data/README.md, PHASE_MATCHED_CONTROL.md) বলে **১১৪**। কিন্তু data থেকে গুনে পাওয়া যায় **১১৫**।

কারণ: script-এর dropped গণনা শুধু "track খুব ছোট" শাখাটা গানে; আরও একজন পথচারী "১৬টা পরপর ফ্রেম নেই" পরীক্ষায় তার সব window হারিয়েছে, যা গানা হয়নি।

করণীয়: নথিতে ১১৫ করে দিন (বা দুইটাই ব্যাখ্যা করুন)। কোনো ফলাফল বদলায় না।

২. গাড়ির গতির ablation-এ দুই ধরনের সংখ্যা মেশানো

CONTRIBUTIONS.md লেখে "+0.0126 থেকে +0.1477"। কিন্তু **0.0126 হলো ensemble bootstrap delta**, আর **0.1477 হলো seed-গাড়ির পার্থক্য** – দুইটা ভিন্ন রাশি।

সঙ্গতিপূর্ণ জোড়া: ensemble **+0.0126 ... +0.1420**, অথবা seed-গাড়ি **+0.0156 ... +0.1477**।

করণীয়: paper-এ যেকোনো একটা বেছে নিয়ে ধারাবাহিকভাবে ব্যবহার করুন।

৩. "সব ৮৫৫ জন non-crosser-এর crossing_point = শেষ ফ্রেম - ২"

LIMITATIONS.md §1-এ "all ... exactly 2" লেখা।

যাচাইয়ে: **855** সংখ্যাটা ঠিক (পুরো annotation-এ non-crossing পথচারী)। কিন্তু যে ৮৩৩ জন পরিষ্কার dataset-এ আসে, তাদের মধ্যে ঠিক ২ হয় **824 জনের (98.9%)**, পরিসর 0-2।

করণীয়: "all ... exactly" নরম করে "98.9%, পরিসর 0-2" লিখুন। **যুক্তি একটুও বদলায় না** – negative-রা এখনো track-এর শেষে নোঙর করা।

৪. Demo-র §9 সংখ্যাগুলো পুনরুৎপাদন হয় না

LIMITATIONS.md §9 বলে: 3,652 ভবিষ্যদ্বাণী, $r = -0.908$, 96.8%, ২০ km/h-এর উপরে 0.45%, "মোলোটা অংশের ছয়টিতে 100%"।

সংরক্ষিত demo ফাইল থেকে আমি পাই: **৪টা ফাইল, 19,799 রেকর্ড, $r = -0.892$, 96.2%, 1.00%**। ৩,৬৫২ রেকর্ডের কোনো স্বাভাবিক উপসেট পাওয়া যায়নি; "মোলোটা অংশ"-এরও চিহ্ন নেই।

করণীয়: সারবস্তু একই (model কার্যত গতির প্রেশহোল্ড), কিন্তু সঠিক সংখ্যাগুলো হয় পুনরায় হিসাব করে লিখুন, নয়তো কোন উপসেটে সেটা নথিভুক্ত করুন। **এই নথিতে আমি নিজের পুনঃহিসাব করা সংখ্যাই ব্যবহার করেছি।**

আরও দুটো জিনিস জেনে রাখা ভালো

ক) পরিষ্কার leakage report-এ একটা ছোট অভ্যন্তরীণ টানাপোড়েন আছে: সে বলে "0টা window-এ crossing ফ্রেম", কিন্তু একই সাথে "1,648 জনের মধ্যে 12 জনের (0.7%) onset window শেষের আগে বা সমানে"। প্রধান দাবিটা (**0/4906**) সরাসরি `cross == "crossing"` গুনে পাওয়া, তাই সেটাই quote করুন।

খ) Zero-shot ব্যর্থতার দাবি rescale রূপান্তরের জন্য সত্য; raw-তে F1 0.244-0.268 (baseline-এর উপরে)। কোনটা quote করছেন **স্পষ্ট করে বলুন**, নইলে reviewer এটাকে cherry-picking বলবে।

এক লাইনে মনে রাখার কথা

Leakage ঠিক করার পরেও data-তে একটা লুকানো সংকেত রয়ে গিয়েছিল – শুধু "negative-দের কখন দেখা হয়েছে" সেটাই। সেটা সরালে সব model প্রায় 0.05 AUC হারায়, আমাদের সেরা model আর সেরা থাকে না, এবং গাড়ির গতির প্রাধান্যের গল্পটা উল্টে যায়।

এজন্যই এটা Limitations-এ লুকানো যাবে না – এটাই paper-এর সবচেয়ে শক্তিশালী প্রমাণ যে প্রথম সমাধানটাই যথেষ্ট নয়।

এই নথির সব সংখ্যা তৈরির সময় যাচাই করা। উৎস ফাইলের তালিকা §১২-এ।